

US002 - Login - Challenge 02

Sendo um vendedor de uma loja com cadastro já realizado

Gostaria de poder me autenticar no Marketplace da ServeRest

Para poder cadastrar, editar, atualizar e excluir meus produtos

DoR

- Banco de dados e infraestrutura para desenvolvimento disponibilizados;
- API de cadastro de usuários implementada;
- Ambiente de testes disponibilizado.

DoD

- Autenticação com geração de token Bearer implementada;
- Análise de testes cobrindo a rota de login;
- Matriz de rastreabilidade atualizada;
- Automação de testes baseado na análise realizada;

Acceptance Criteria

- Usuários não cadastrados não deverão conseguir autenticar;
- Usuários com senha inválida não deverão conseguir autenticar;
- No caso de não autenticação, deverá ser retornado um status code 401 (Unauthorized);
- Usuários existentes e com a senha correta deverão ser autenticados;
- A autenticação deverá gerar um token Bearer;
- A duração da validade do token deverá ser de 10 minutos;
- Os testes executados deverão conter evidências;
- A cobertura de testes deve se basear no Swagger e ir além, cobrindo cenários alternativos.

Cenários de Teste [🔗](#)

CT-015 – Login com credenciais válidas [🔗](#)

Pré-condição: Usuário válido cadastrado (ex: joao@valido.com, senha 123456)

Passos:

1. Realizar uma requisição POST para o endpoint /login
2. Enviar o seguinte payload:

```
1 {  
2   "email": "joao@valido.com",  
3   "password": "123456"  
4 }
```

Resultado Esperado:

- Status code 200
 - Mensagem: "Login realizado com sucesso"
 - Token do tipo Bearer é retornado
-

CT-016 – Login com usuário não cadastrado [↗](#)

Pré-condição: E-mail não existe na base

Passos:

1. Realizar uma requisição POST para `/login` com um e-mail inexistente
2. Enviar senha válida qualquer

Resultado Esperado:

- Status code `401`
 - Mensagem: "Email e/ou senha inválidos"
-

CT-017 – Login com senha inválida [↗](#)

Pré-condição: E-mail existente com senha correta na base

Passos:

1. Realizar POST para `/login` com senha incorreta

Resultado Esperado:

- Status code `401`
 - Mensagem: "Email e/ou senha inválidos"
-

CT-018 – Login sem fornecer senha [↗](#)

Passos:

1. Realizar POST para `/login` omitindo o campo `"password"`

Resultado Esperado:

- Status code `400`
 - Mensagem de erro indicando campo obrigatório ausente
-

CT-019 – Login sem fornecer e-mail [↗](#)

Passos:

1. Realizar POST para `/login` omitindo o campo `"email"`

Resultado Esperado:

- Status code `400`
 - Mensagem de erro indicando campo obrigatório ausente
-

CT-020 – Validade do token (10 minutos) [↗](#)

Pré-condição: Usuário autenticado com token gerado

Passos:

1. Fazer login e guardar token
2. Esperar 10 minutos
3. Realizar GET em endpoint protegido (ex: `/produtos`) com esse token

Resultado Esperado:

- Após 10 minutos, status code 401 Unauthorized
- Mensagem indicando expiração do token

Pós-condição: Sessão expirada

CT-021 – Reutilização de token após logout ou expiração

Pré-condição: Usuário autenticado, token expirado ou invalido após logout

Passos:

1. Tentar acessar /produtos com token expirado ou inválido

Resultado Esperado:

- Status code 401 Unauthorized

Priorização dos Testes

ID	Endpoint	Cenário	Tipo	Prioridade	Status
CT-015	/login	Login com e-mail e senha corretos	Funcional / Automatizado	Crítica	NÃO EXECUTADO
CT-016	/login	Login com usuário inexistente	Funcional / Automatizado	Crítica	NÃO EXECUTADO
CT-017	/login	Login com senha incorreta	Funcional / Automatizado	Crítica	NÃO EXECUTADO
CT-018	/login	Login sem campo password	Funcional / Automatizado	Média	NÃO EXECUTADO
CT-019	/login	Login sem campo email	Funcional / Automatizado	Média	NÃO EXECUTADO
CT-020	/login	Token expira após 10 minutos	Funcional / Automatizado	Crítica	NÃO EXECUTADO
CT-021	/login	Reutilização de token após logout/expiração	Funcional / Automatizado	Crítica	NÃO EXECUTADO